PYTHON VS JAVA

The Python Code (speed_test.py)

```
import time

def main():
    start_time = time.time()

    count = 0
    max_value = 100_000_000  # 100 Million

    while count < max_value:
        count += 1

    end_time = time.time()
    elapsed_time = end_time - start_time

    print(f"Python finished in: {elapsed_time:.4f} seconds")

if __name__ == "__main__":
    main()
```

OUTPUT: Python finished in: 3.4955 seconds

JAVA

```
public class SpeedTest {
    public static void main(String[] args) {
        long startTime = System.nanoTime();
```

```
        int count = 0;
        int maxValue = 100_000_000; // 100 Million

        while (count < maxValue) {
            count++;
        }

        long endTime = System.nanoTime();
        // Convert nanoseconds to seconds
        double elapsedTime = (endTime - startTime) / 1_000_000_000.0;

        System.out.printf("Java finished in: %.4f seconds\n", elapsedTime);
    }
}
```

OUTPUT : Java finished in: 0.0627 seconds

REASONS

1. Dynamic vs. Static Typing

- Python has to check the data type of count and max_value on every single iteration of the loop to ensure they are still numbers. This constant checking creates massive CPU overhead.
- Java is statically typed, meaning it defines the variable type *before* the loop starts. Because the computer already knows the data type will never change, it skips all type-checking during execution.

2. Interpretation vs. JIT Compilation

- Python reads, interprets, and executes the loop line-by-line, repeating the translation process for all 100 million cycles.

- Java uses a Just-In-Time (JIT) Compiler. When it detects a heavy loop, it compiles that specific block directly into native machine code, allowing the CPU to run the remaining cycles at hardware speed.

3. Object Overhead vs. Primitive Types

- In Python, integers are heavy, complex objects that require memory allocation and unwrapping for every mathematical operation.
- In Java, variables can be stored as primitive types (int), which are raw numbers processed directly inside the CPU registers with zero memory overhead.